

1 Sorting and searching**1.1 Insertion sort**

Take elements from right to left, insert them in the right sorted portion of the list. This algorithm sorts **in-place**.

Time complexity:

- Best: $\Theta(n)$; already sorted array
- Average: $\Theta(n^2)$
- Worst: $\Theta(n^2)$; array sorted in reverse

Space complexity: Recursion depth = n ; re-uses the tail list.

- Best / Average / Worst: $\Theta(n)$

```
1 function insert(x: number, xs: List<number>): List<number> {
2   if (is_null(xs)) {
3     return list(xs);
4   } else if (x <= head(xs)) {
5     // found correct position, resolve
6     return pair(x, xs);
7   } else {
8     // move element ahead
9     return pair(head(xs), insert(x, tail(xs)));
10  }
11 }
12
13 function insertion_sort(xs: List<number>): List<number> {
14   return is_null(xs)
15     ? xs
16     : insert(head(xs), insertion_sort(tail(xs)));
17 }
```

1.2 Selection sort

Choose the smallest element in a list. Place that element in order in a new list.

Time complexity: Number of comparisons is fixed.

- Best/Average/Worst: $\Theta(n^2)$

Space complexity: Recursion depth = n ; remove() creates an n sized copy.

- Best/Average/Worst: $\Theta(n^2)$

```
1 function smallest(xs: List<number>): number {
2   function h(xs: List<number>, min: number): number {
3     return is_null(xs)
4       ? min
5       : head(xs) < min
6         ? h(tail(xs), head(xs))
7         : h(tail(xs), min);
8   }
9   return h(xs, head(xs));
10 }
11
12 function selection_sort(xs: List<number>): List<number> {
13   if (is_null(xs)) {
14     return xs;
15   } else {
16     const s = smallest(xs);
17     // smallest elem s goes in front of the list
18     return pair(s, selection_sort(remove(s, xs)));
19   }
20 }
21
22 }
```

1.3 Quick sort

A divide-and-conquer algorithm. Choose a pivot, and partition the remaining elements into 2 groups: smaller and larger. Then recursively partition the groups: quicksort(smaller) + [pivot] + quicksort(larger).

Time complexity:

- Best: $\Theta(n \log n)$
- Average: $\Theta(n \log n)$
- Worst: $\Theta(n^2)$; pivot always unbalanced, ie. smaller $\ll$ bigger or bigger $\gg$ smaller

Space complexity:

- Best: $\Theta(n \log n)$
- Average: $\Theta(n \log n)$; recursion depth = $\log n$.
- Worst: $\Theta(n^2)$; pivot always unbalanced, recursion depth = n .

```
1 function partition(xs: List<number>, pivot: number): Pair<List<number>,
2   List<number>> {
3   return accumulate(
4     (curr, acc) => {
5       if (curr <= xs) {
6         return pair(
7           pair(curr, head(acc)),
8           tail(acc)
9         );
10      } else {
11        return pair(
12          head(acc),
13          pair(curr, tail(acc))
14        );
15      }
16    }, pair(list(), list()), // 0 is smaller, 1 is larger
17    xs
18  );
19 }
20
21 function quicksort(xs: List<number>): List<number> {
22   if (is_null(xs)) {
23     return list(null);
24   } else {
25     const pivot = head(xs);
26     const s = partition(tail(xs), pivot);
```

```
27     return append(
28       quicksort(head(s)), // smaller
29       append(
30         list(pivot),
31         quicksort(tail(s)) // larger
32       );
33   }
34 }
35 }
```

1.4 Merge sort

Divide and conquer. Recursively split unsorted elements and then merge them back together sorted.

Note: see utilities for implementation of take() and drop().

Time complexity: Splitting creates $\log n$ problems of merging lists of size n

- Best/Average/Worst: $\Theta(n \log n)$

Space complexity: Merging creates a list with total size n

- Best/Average/Worst: $\Theta(n)$

```
1 function merge(xs: List<number>, ys: List<number>): List<number> {
2   // base cases: run out of elements in one of the lists, just append the
3   // other list
4   if (is_null(xs)) {
5     return ys;
6   } else if (is_null(ys)) {
7     return xs;
8   }
9
10  function merge_sort(xs: List<number>): List<number> {
11    if (is_null(xs) || is_null(tail(xs))) {
12      return xs;
13    } else {
14      const mid = math_floor(length(xs) / 2);
15      return merge(
16        // continue splitting until base case
17        merge_sort(take(xs, mid)),
18        merge_sort(drop(xs, mid))
19      );
20    }
21  }
22 }
```

1.5 Binary search

Assume sorted list.

Time complexity: list_ref() is an $O(n)$ operation.

- Best: $\Theta(n)$; target in the middle
- Average: $\Theta(n)$
- Worst: $\Theta(n)$; $\log n$ iterations

Space complexity: $\log n$ iterations asymptotically.

- Best/Average/Worst: $\Theta(\log n)$

```
1 function binary_search(xs, target) {
2   function helper(low, high) {
3     if (low > high) {
4       return -1;
5     } else {
6       const mid = math_floor((low + high) / 2);
7       const v = list_ref(xs, mid);
8
9       if (v === target) {
10        return mid;
11      } else if (target < v) {
12        return helper(low, mid - 1); // search left
13      } else {
14        return helper(mid + 1, high); // search right
15      }
16    }
17  }
18  return helper(0, length(xs) - 1);
19 }
```

2 Permutations and Combinations**2.1 Permutations**

Get permutations of a list. Order matters.

```
1 function permutations<T>(xs: List<T>): List<List<T>> {
2   if (is_null(xs)) {
3     return list(null);
4   } else {
5     // Flatten list [[1,2], [[2, 1]] => [[1,2], [2, 1]]
6     // x is prepended to all possible permutations
7     return accumulate(
8       append(
9         list(),
10        map(
11          // choose x from xs
12          x => map(
13            // prepend x to the permutations of remaining
14            p => pair(x, p),
15            // recursively generate all permutations of remaining list
16            permutations(remove(x, xs))
17          ),
18          xs
19        );
20      ),
21      xs
22    );
23  }
```

2.2 Combinations

Get combinations of sub-lists (length r) of a list.

```
1 // 0 <= r <= n
```

```
2 function combinations<T>(xs: List<T>, r: number): List<List<T>> {
3   if (r === 0) {
4     return list(null);
5   } else if (is_null(xs)) {
6     return null;
7   } else {
8     // we do not choose the head(xs)
9     const no = combinations(tail(xs), r);
10    // choose the head(xs) and prepend it to possible combinations of
11    // length r - 1
12    const yes = map(
13      x => pair(head(xs), x),
14      combinations(tail(xs), r - 1)
15    );
16    return append(no, yes);
17  }
18 }
```

2.3 N Choose R

```
1 // Formula: nCr = n! / (r! * (n-r)!)
2 // Recursive: nCr = (n-1)C(r-1) + (n-1)C(r) (Pascal's triangle)
3 // Example: 5C3 = 4C2 + 4C3 = 6 + 4 = 10
4 function nCr(n, r) {
5   if (r > n || r < 0) return 0;
6   if (r === 0) return 1;
7
8   // choose + dont choose
9   return nCr(n - 1, r - 1) + nCr(n - 1, r);
10 }
```

2.4 N Permute R

```
1 // Formula: nPr = n! / (n-r)!
2 // Recursive: nPr = n * (n-1)P(r-1)
3 // Example: 5P3 = 5 * 4P2 = 5 * 4 * 3P1 = 5 * 4 * 3 = 60
4 function nPr(n, r) {
5   if (r > n || r < 0) return 0;
6   if (r === 0) return 1;
7
8   // return n * nPr(n - 1, r - 1);
9 }
10 }
```

2.5 Subsets

Get all possible subsets of xs. Modified version of combinations.

```
1 function subsets<T>(xs: List<T>): List<List<T>> {
2   if (is_null(xs)) {
3     return list(null);
4   } else {
5     const no = subsets(tail(xs));
6     const yes = map(
7       // prepend all head elems onto the remainder
8       x => pair(head(xs), x),
9       subsets(tail(xs))
10    );
11    return append(no, yes);
12  }
13 }
```

3 Binary Trees

Definition: left sub-tree < node < right sub-tree

3.1 Insert

```
1 function insert<T extends number | string>(bst: Tree<T>, item: T): Tree<T> {
2   if (is_empty_tree(bst)) {
3     return make_tree(item, make_empty_tree(), make_empty_tree());
4   } else if (item < entry(bst)) {
5     // smaller: go left
6     return make_tree(entry(bst),
7       insert(left_branch(bst), item),
8       right_branch(bst));
9   } else if (item > entry(bst)) {
10    // bigger: go right
11    return make_tree(entry(bst),
12      left_branch(bst),
13      insert(right_branch(bst), item));
14   } else {
15     // equal: no duplicates. immediately return.
16     return bst;
17   }
18 }
```

3.2 Search

Check if element in tree. Assume no duplicates.

```
1 function search<T extends number | string>(bst: Tree<T>, target: T):
2   boolean {
3   if (is_empty_tree(bst)) {
4     return false;
5   } else if (entry(bst) === target) {
6     return true;
7   } else if (target < entry(bst)) {
8     // smaller, search left
9     return search(left_branch(bst), target);
10  } else {
11    // larger, search right
12    return search(right_branch(bst), target);
13  }
14 }
```

3.3 Traverse

- Pre-order: root, left, right
- In-order: left, root, right
- Post-order: left, right, root

```
1 function append3<T extends List<unknown>>(a: T, b: T, c: T): List<T> {
```

```
2   return append(
3     a, append(b, c)
4   );
5 }
6
7 function preorder<T extends number | string>(bst: Tree<T>): List<T> {
8   if (is_empty_tree(bst)) {
9     return make_empty_tree();
10  } else {
11    return append3(
12      list(entry(bst)),
13      preorder(left_branch(bst)),
14      preorder(right_branch(bst))
15    );
16  }
17 }
18
19 function inorder<T extends number | string>(bst: Tree<T>): List<T> {
20   if (is_empty_tree(bst)) {
21     return make_empty_tree();
22   } else {
23     return append3(
24       inorder(left_branch(bst)),
25       list(entry(bst)),
26       inorder(right_branch(bst))
27     );
28   }
29 }
30
31 function postorder<T extends number | string>(bst: Tree<T>): List<T> {
32   if (is_empty_tree(bst)) {
33     return make_empty_tree();
34   } else {
35     return append3(
36       postorder(left_branch(bst)),
37       postorder(right_branch(bst)),
38       list(entry(bst))
39     );
40   }
41 }
```

3.4 Delete

```
1 function delete<T extends number | string>(bst: Tree<T>, target: T):
2   Tree<T> {
3   if (is_empty_tree(bst)) {
4     return bst; // nothing to delete
5   } else if (target < entry(bst)) {
6     // smaller: go left
7     return make_tree(
8       entry(bst),
9       delete(left_branch(bst), target),
10      right_branch(bst)
11    );
12  } else if (target > entry(bst)) {
13    // bigger: go right
14    return make_tree(
15      entry(bst),
16      left_branch(bst),
17      delete(right_branch(bst), target)
18    );
19  } else {
20    // found node to delete
21    if (is_empty_tree(left_branch(bst))) {
22      return right_branch(bst); // only right child or none
23    } else if (is_empty_tree(right_branch(bst))) {
24      return left_branch(bst); // only left child
25    } else {
26      // two children: replace with inorder successor
27      // successor is leftmost number of the right branch
28      const successor = find_min(right_branch(bst));
29      return make_tree(
30        successor,
31        left_branch(bst),
32        // remove the smallest number in the right branch
33        delete(right_branch(bst), successor)
34      );
35    }
36  }
37 }
38
39 // find the smallest number in tree
40 function find_min<T extends number | string>(bst: Tree<T>): T {
41   if (is_empty_tree(left_branch(bst))) {
42     return entry(bst);
43   } else {
44     return find_min(left_branch(bst));
45   }
46 }
```

3.5 Invert

Swap the left and right sub-trees of every node in the tree.

```
1 function invert<T extends number | string>(bst: Tree<T>): Tree<T> {
2   if (is_empty_tree(bst)) {
3     return bst;
4   } else {
5     return make_tree(
6       entry(bst),
7       invert(right_branch(bst)), // swap sides
8       invert(left_branch(bst))
9     );
10  }
11 }
```

3.6 Flatten

```
1 function flatten_tree<T>(tree: Tree<T>): List<T> {
2   return accumulate(
3     (x, y) => is_list(x)
```

```

4      ? append(flatten(x), y)
5      : pair(x, y),
6      null,
7      tree);
8  }

```

3.7 Map

```

1  function map_tree<T, U>(f: (x: T) => U, tree: Tree<T>): Tree<U> {
2      return map(
3          x => is_list(x)
4              ? map_tree(f, x)
5              : f(x),
6          tree);
7  }

```

3.8 Accumulate/Reduce

```

1  function accumulate_tree<T, U>(f: (x: T, y: U) => U, init: U, tree:
2  Tree<T>): U {
3      return accumulate(
4          (x, y) => is_list(x)
5              ? accumulate_tree(f, y, x)
6              : f(x, y),
7          init,
8          tree);
9  }

```

4 Tombstone diagrams

4.1 Interpreter

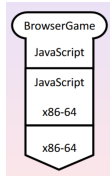


Figure 1: Interpreting JS (browser runtime)

4.2 Compiler

T-figure: Language 1 (left) translates to language 2 (right) via compiler written with language 3 (bottom).

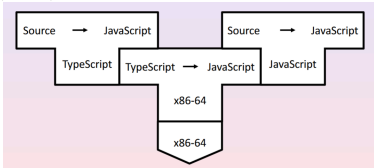


Figure 2: Compiling a compiler

5 Space complexity, Time complexity

5.1 Order of growth

Big Theta: The function r has order of growth $\Theta(g(n))$ if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \cdot g(n) \leq r(n) \leq k_2 \cdot g(n)$ for any $n > n_0$.

Big O: The function r has order of growth $O(g(n))$ if there is a positive constant k such that $r(n) \leq k \cdot g(n)$ for any sufficiently large value of n .

Big Omega: The function r has order of growth $\Omega(g(n))$ if there is a positive constant k such that $k \cdot g(n) \leq r(n)$ for any sufficiently large value of n .

Order (small to big): 1, $\log n$, n , $n \log n$, n^k , k^n , n^n

5.2 Tips for analysis

Recurrence Notation: $T(n)$ represents the time complexity for input size n . The $+$ separates the work done at the current level from the recursive calls. For example, $T(n) = \Theta(1) + 2T(n/2)$ means constant work at this level, plus two recursive calls on half-sized problems. The coefficient before T indicates the number of recursive calls (e.g., $kT(n-1)$ means k recursive calls).

Case	Recurrence / Pattern	Complexity
Linear scan	$T(n) = \Theta(1) + T(n-1)$	$\Theta(n)$
Nested loops (k loops)	–	$\Theta(n^k)$
Divide & conquer (typical)	$T(n) = \Theta(n) + 2T(n/2)$	$\Theta(n \log n)$
Subproblems (k per step)	$T(n) = kT(n-1)$	$\Theta(k^n)$

Recursive calls to $f(n-k)$	$T(n) = \Theta(1) + T(n-a)$	$\Theta(n)$
Recursive calls to $f(n-k)$ with linear work	$T(n) = \Theta(n) + T(n-a)$	$\Theta(n^2)$
Recursive calls to $f(n/k)$	$T(n) = \Theta(1) + T(n/a)$	$\Theta(\log n)$
Recursive calls to $f(n/k)$ with linear work	$T(n) = \Theta(n) + T(n/a)$	$\Theta(n)$
Branching recursion (b sub-problems of size $n-a$)	$T(n) = \Theta(1) + bT(n-a)$	$\Theta(c^n)$
Branching recursion with linear work	$T(n) = \Theta(n) + bT(n-a)$	$\Theta(c^n)$
Divide & conquer with constant work	$T(n) = \Theta(1) + aT(n/a)$	$\Theta(n)$
Divide & conquer with linear work	$T(n) = \Theta(n) + aT(n/a)$	$\Theta(n \log n)$
Work decreases per recursive call	$T(n) = g(i)$ per level, depth = d	$\Theta(\sum_{i=0}^d g(i))$

6 Utilities

6.1 Definitions

Tree: A tree of a certain data type is a list whose elements are of that data type, or trees of that data type.

```
1 type Tree<T> = List<T | Tree<T> | null>
```

List: A list is either null or a pair whose tail is a list.

6.2 Take/Drop

6.2.1 Take

Take the first n elements.

```

1 function take<T>(xs: List<T>, n: number): List<T> {
2     return n == 0
3         ? null
4         : pair(head(xs), take(tail(xs), n - 1));
5 }

```

6.2.2 Drop

Drop the first n elements.

```

1 function drop<T>(xs: List<T>, n: number): List<T> {
2     return n == 0
3         ? xs
4         : drop(tail(xs), n - 1);
5 }

```

6.3 Unique

Get all unique elements.

```

1 function unique<T>(xs: List<T>): List<T> {
2     return is_null(xs)
3         ? null
4         : append(
5             list(head(xs)),
6             unique(remove_all(head(xs), tail(xs)))
7         );
8 }

```

6.4 Sinusoidal transformations (for sound)

For $f(x) = a \sin(bx + c) + d$

6.4.1 Amplitude

- a : amplitude scaling factor
- $|a| > 1$: stretch, $0 < |a| < 1$: compress
- $a < 0$: reflect across x-axis

6.4.2 Period

- b : period scaling factor
- $|b| > 1$: compress, $0 < |b| < 1$: stretch
- Period = $\frac{2\pi}{|b|}$

6.4.3 Phase Shift

- c : horizontal shift factor
- Phase shift = $-\frac{c}{b}$ units
- $c > 0$: left shift, $c < 0$: right shift

6.4.4 Vertical Translation

- d : vertical shift
- $d > 0$: up, $d < 0$: down

6.5 Parametric Curve Transformations (for curves)

For curve $x = f(t)$, $y = g(t)$ where t is the parameter.

6.5.1 Scaling

$x = k_x \cdot f(t)$, $y = k_y \cdot g(t)$

- k_x : horizontal scaling factor
- k_y : vertical scaling factor
- $k_x, k_y > 1$: stretch
- $0 < k_x, k_y < 1$: compress
- $k_x, k_y < 0$: reflect and scale

6.5.2 Translation

$x = f(t) + h$, $y = g(t) + k$

- h : horizontal shift (positive = right)
- k : vertical shift (positive = up)

6.5.3 Direction Reversal

$x = f(-t)$, $y = g(-t)$

- $t \rightarrow -t$: reverses curve orientation

6.6 Evaluating SKI combinatorics problems

Rules:

- $(Kab) \rightarrow a$
- $(Ix) \rightarrow x$
- $(Sfgx) \rightarrow (fx)(gx)$

Tips:

- S requires 3 args, K requires 2 args, I requires 1 arg.
- Scan left to right. Each combinator "expects" a certain number of arguments. As soon as it is satisfied, reduce.
- Reduce innermost complete groups. eg. $S(KI)Kx = (KIx)(Kx) = I(Kx) = Kx$

6.7 List shifting

6.7.1 Shift element from end to start

```

1 function shift<T>(xs: List<T>): List<T> {
2     return is_null(xs)
3         ? null
4         : accumulate(pair, list(head(xs)), tail(xs));
5 }

```

6.7.2 Shift element from start to end

```

1 function shift<T>(xs: List<T>): List<T> {
2     if (is_null(tail(xs)) || is_null(xs)) {
3         return xs;
4     } else {
5         const wish = shift(tail(xs));
6         return pair(head(wish), pair(head(xs), tail(wish)));
7     }
8 }

```

6.8 Active Lists (slices)

Kind of like a Golang slice. `make_active_list(xs: List<unknown>)` returns a function with 1 argument. Calling this function with argument n , $0 \leq n \leq \text{length}(xs) - 1$ results in `xs[n]`, otherwise null.

6.8.1 Length

Recover length of list backing active list.

```

1 function length<T>(as: ActiveList<T>): number {
2     function h(as: ActiveList<T>, count: number): number {
3         return is_null(as(count))
4             ? 0
5             : 1 + h(as(count), count + 1);
6     }
7     return h(as, 0);
8 }

```

6.8.2 Delete

Delete `as[pos]` from `as`, resulting in a active list that is length $n-1$.

```

1 function remove<T>(as: ActiveList<T>, pos: number): ActiveList<T> {
2     return p => p < pos ? A(p) : A(p + 1);
3 }

```

6.8.3 Append

Append active list as to bs.

Note: see active list length function in earlier section.

```

1 function append<T>(as: ActiveList<T>, bs: ActiveList<T>): ActiveList<T> {
2     // as has indexes from 0...(length(as) - 1)
3     return p => p < length(as) ? as(p) : bs(p);
4 }

```

6.9 Vectors

6.9.1 Equality

```

1 function equal(x: List<number>, y: List<number>): boolean {
2     return is_pair(x) && is_pair(y)
3         ? head(x) == head(y) && equal(tail(x), tail(y))
4         : // we have reached the end, assert length equality
5           is_null(x) && is_null(y);
6 }

```

6.9.2 Dot product

Assume equal length vectors.

```

1 function dot(x: List<number>, y: List<number>): number {
2     return is_null(x)
3         ? 0
4         : head(x) * head(y) + dot(tail(x), tail(y));
5 }

```

6.9.3 Distance between vectors

Assume equal length vectors. Formula is $d_w = \sqrt{\sum_{n=0}^{\text{dim}(v)} (x_n - y_n)^2}$

```

1 function dist(x: List<number>, y: List<number>): number {
2     const square = x => x * x; // for some fuckass reason pow()/** operator
3     is not available in source s2
4
5     function h(x: List<number>, y: List<number>): number {
6         return is_null(x)
7             ? 0
8             : square(head(x) - head(y)) + h(tail(x), tail(y));
9     }
10    return math_sqrt(h(x, y));
11 }

```

6.10 Matrices

Represented as `List<List<T>>`.

6.10.1 Getting element

```

1 function get_elem<T>(M: List<List<T>>, r: number, c: number): T {
2     return list_ref(list_ref(M, r), c);
3 }

```

6.10.2 Transposition

Flip rows and columns.

```

1 function transpose(M: List<List<T>>): List<List<T>> {
2     const nR = length(M); // number of rows
3     const nC = length(head(M)); // number of columns
4
5     return map( c => map(r => get_elem(M, r, c), enum_list(0, nR - 1)) ,
6         enum_list(0, nC - 1) );
7 }

```

6.10.3 Addition

```

1 function add_matrices<T extends number>(
2     M1: List<List<T>>,
3     M2: List<List<T>>
4 ): List<List<T>> {
5     const nR = length(M1);
6     const nC = length(head(M1));
7
8     return map(
9         r => map(
10            c => get_elem(M1, r, c) + get_elem(M2, r, c),
11            enum_list(0, nC - 1)
12        ),
13        enum_list(0, nR - 1)
14    );
15 }

```

6.10.4 Multiplication

Assume M1 and M2 have n rows and n columns respectively.

Note: see vector dot().

```

1 function multiply_matrices(
2     M1: List<List<number>>,
3     M2: List<List<number>>
4 ): List<List<number>> {
5     const M2T = transpose(M2);
6     return map(
7         row => map(col => dot(row, col), M2T),
8         M1
9     );
10 }

```

6.11 Continuation-Passing Style

Motivation: A programming pattern where we convert recursive functions into iterative ones by passing the "rest of the computation" as a function argument. This deferred operation (the continuation) accumulates the work to be done after the current step completes. CPS can convert any recursive function into an iterative form.

Recursive version:

```

1 function factorial(n: number): number {
2     return n == 1
3         ? 1
4         : n * factorial(n - 1);
5 }

```

CPS version (iterative):

```

1 function fac<T>(n: number, c: (x: number) => T): T {
2     return n == 1
3         ? c(1)
4         : fac(n - 1, x => c(n * x));
5 }
6
7 function factorial_iter(n: number): number {
8     return fac(n, x => x);
9 }

```

The continuation `c` accumulates the pending multiplications. Instead of building up a call stack, we build up a chain of functions that will be applied once we reach the base case.